



INSTITUT NATIONAL SUPERIEUR DES
SCIENCES ET TECHNIQUES D'ABECHE

RAPPORT DE PROJET PROFESSIONNELLE ET PERSONNELLE

*3ème Année en Génie Informatique
Option Génie Logiciel*

IA pour la conversation de schéma UML en code fonctionnel

Réalisé par :

Abakar Mahmat Brahim

Encadreur (s) :

Encadrant Professionnel : Dr. MAHAMAT HABIB

Encadrant Pédagogique : M. ABOUBAKAR

Année universitaire : 2025/206

Dédicace

À mes parents,

*pour leur soutien indéfectible, leurs sacrifices et leur amour inconditionnel
tout au long de mon parcours académique.*

À mes frères et sœurs,

pour leur encouragement et leur présence à chaque étape de ma vie.

À mes enseignants de l'INSTA Abéché,

*pour la qualité de la formation dispensée et leur dévouement
au service de la jeunesse tchadienne.*

*À tous ceux qui croient que la technologie
peut transformer le continent africain.*

Je dédie ce travail.

Remerciements

Au terme de ce travail, il m'est agréable d'exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Mes premiers remerciements s'adressent à **Dr. MAHAMAT HABIB**, mon encadrant professionnel, pour sa disponibilité, ses conseils avisés et son suivi tout au long de ce projet. Ses orientations techniques ont été déterminantes dans la réussite de ce travail.

Je remercie également **M. ABOUBAKAR**, mon encadrant pédagogique, pour son soutien académique, ses précieuses remarques et sa rigueur scientifique qui ont grandement contribué à améliorer la qualité de ce rapport.

Je tiens à exprimer ma reconnaissance à l'ensemble du corps enseignant de l'**Institut National Supérieur des Sciences et Techniques d'Abéché (INSTA)**, et plus particulièrement au département de **Génie Logiciel**, pour la qualité de la formation reçue durant ces trois années d'études.

Je remercie sincèrement **l'équipe Groq** pour avoir mis à disposition une API d'inférence IA de haute performance, ainsi que la communauté **open source** de React.js, Flask et Monaco Editor, dont les outils ont été indispensables à la réalisation de ce projet.

Enfin, je remercie chaleureusement **ma famille** pour son soutien moral constant, sa patience et ses encouragements qui ont été une source de motivation inépuisable tout au long de ce travail.

*Abakar Mahamat Brahim
Abéché, Mai 2026*

Résumé

Ce projet présente la conception et le développement d'**UML-to-Code**, une application web full-stack permettant de transformer automatiquement des diagrammes de classes UML en code source fonctionnel, en exploitant l'Intelligence Artificielle générative **Groq Llama 3.3-70b**¹.

L'application accepte trois formats d'entrée : les fichiers **XMI** exportés depuis StarUML, les **images** PNG/JPG et les **documents PDF**. Elle génère du code **Python** ou **PHP** complet avec classes, attributs, méthodes et héritage, ainsi qu'un **serveur Flask** opérationnel avec routes CRUD automatiques.

L'interface web, développée avec **React.js** et Vite, intègre le **Monaco Editor**, un historique local, un mode sombre/clair, une barre de progression et un export ZIP. Déploiement².

Le projet a été réalisé selon le **modèle incrémental** en 5 incréments sur trois semaines, dans le cadre du module de Mini-Projet de Soutenance de la 3^{ème} année en Génie Logiciel à l'INSTA Abéché, Tchad.

Mots-clés : UML, génération de code, Intelligence Artificielle, LLM, Groq, Flask, React.js, MDD, Python, PHP, StarUML, XMI.

1. API Groq : <https://console.groq.com/docs>

2. Vercel : <https://uml-to-code-peach.vercel.app> ; Render : <https://uml-to-code.onrender.com>

Abstract

This project presents the design and development of **UML-to-Code**, a full-stack web application that automatically transforms UML class diagrams into functional source code, leveraging the **Groq Llama 3.3-70b** generative Artificial Intelligence model ³.

The application accepts three input formats : **XMI** files exported from StarUML, **PNG/JPG images**, and **PDF documents**. It generates complete **Python** or **PHP** code with classes, attributes, methods and inheritance, as well as an operational **Flask server** with automatic CRUD routes.

The web interface, built with **React.js** and Vite, integrates Monaco Editor, local history, themes, progress bar and ZIP export. Online deployment ⁴.

The project was carried out using the **incremental model** in 5 increments over three weeks, as part of the Final Project module of the 3rd year Software Engineering program at INSTA Abéché, Chad.

Keywords : UML, code generation, Artificial Intelligence, LLM, Groq, Flask, React.js, MDD, Python, PHP, StarUML, XMI.

3. Groq documentation : <https://console.groq.com/docs>

4. <https://uml-to-code-peach.vercel.app>

Table des matières

Liste des figures

Liste des tableaux

Liste des Sigles et Acronymes

API	<i>Application Programming Interface</i> — Interface de programmation
CORS	<i>Cross-Origin Resource Sharing</i> — Partage de ressources entre origines
CRUD	<i>Create, Read, Update, Delete</i> — Opérations de base sur les données
CSS	<i>Cascading Style Sheets</i> — Feuilles de style en cascade
ENV	<i>Environment</i> — Fichier de variables d'environnement
HTML	<i>HyperText Markup Language</i> — Langage de balisage hypertexte
HTTP	<i>HyperText Transfer Protocol</i> — Protocole de transfert hypertexte
IA	Intelligence Artificielle
INSTA	Institut National Supérieur des Sciences et Techniques d'Abéché
JSON	<i>JavaScript Object Notation</i> — Format d'échange de données
LLM	<i>Large Language Model</i> — Grand modèle de langage
MDA	<i>Model-Driven Architecture</i> — Architecture dirigée par les modèles
MDD	<i>Model-Driven Development</i> — Développement dirigé par les modèles
OMG	<i>Object Management Group</i> — Consortium de standardisation
PHP	<i>PHP : Hypertext Preprocessor</i> — Langage de programmation web
PIM	<i>Platform Independent Model</i> — Modèle indépendant de la plateforme
POO	Programmation Orientée Objet
PSM	<i>Platform Specific Model</i> — Modèle spécifique à la plateforme
REST	<i>Representational State Transfer</i> — Style d'architecture pour API web
SPA	<i>Single Page Application</i> — Application web monopage
UML	<i>Unified Modeling Language</i> — Langage de modélisation unifié
URL	<i>Uniform Resource Locator</i> — Adresse web
Vite	Outil de build frontend pour React.js
Groq	Plateforme d'inférence IA (modèle Llama 3.3)
XMI	<i>XML Metadata Interchange</i> — Format d'échange de modèles UML
XML	<i>eXtensible Markup Language</i> — Langage de balisage extensible
ZIP	<i>Zone Information Protocol</i> — Format d'archive compressée

Introduction Générale

Contexte et problématique

Dans le domaine du Génie Logiciel, la modélisation UML⁵ (*Unified Modeling Language*) constitue une étape incontournable du cycle de développement logiciel. Elle permet aux développeurs et architectes de concevoir, documenter et communiquer l'architecture d'un système de manière standardisée, avant même d'écrire la première ligne de code. Cependant, le passage du modèle conceptuel UML au code source représente encore aujourd'hui une tâche manuelle, répétitive et souvent source d'erreurs et d'incohérences.

Cette problématique s'inscrit dans le cadre du **Développement Dirigé par les Modèles** (MDD — *Model-Driven Development*), un paradigme qui préconise la génération automatique de code à partir de modèles de haut niveau. Malgré l'existence de plusieurs outils de transformation UML vers code, ceux-ci restent soit trop complexes à installer et à utiliser, soit limités à un seul langage cible, soit dépourvus de toute intelligence artificielle pour produire un code contextuel et exploitable directement.

Motivation

L'émergence des modèles d'Intelligence Artificielle générative de grande taille (*Large Language Models* — LLM) ouvre de nouvelles perspectives pour automatiser et enrichir ce processus de transformation. En exploitant la capacité de ces modèles à comprendre la structure d'un diagramme UML et à générer du code source cohérent, il devient possible de franchir un cap significatif dans l'automatisation du développement logiciel.

C'est dans ce contexte que s'inscrit le présent projet, réalisé dans le cadre du module de Mini-Projet de Soutenance de la troisième année en Génie Logiciel à l'Institut National Supérieur des Sciences et Techniques d'Abéché (INSTA), Tchad.

Objectifs

Ce projet a pour objectif principal de concevoir et développer une application web full-stack, baptisée **UML-to-Code**, capable de transformer automatiquement un diagramme de classes UML en code source fonctionnel, en exploitant le modèle d'IA générative **Groq Llama 3.3-70b**⁶.

Plus précisément, les objectifs visés sont les suivants :

- Accepter plusieurs formats d'entrée : fichier XMI exporté depuis StarUML, image PNG/JPG, ou document PDF
- Générer du code source **Python** ou **PHP** complet avec classes, attributs, méthodes et relations d'héritage
- Générer automatiquement un **serveur Flask** avec les routes CRUD correspondantes
- Proposer une interface web moderne, intuitive et responsive avec un éditeur de code intégré (**Monaco Editor**)
- Permettre l'export du projet complet en archive **ZIP** prête à déployer
- Déployer l'application en ligne⁷ et la rendre accessible au grand public

5. UML — OMG : <https://www.omg.org/spec/UML/>

6. API Groq : <https://console.groq.com/docs>

Structure du rapport

Ce rapport est organisé en trois chapitres principaux, encadrés par une introduction et une conclusion générales :

- **Chapitre 1 — Étude préliminaire et analyse des besoins** : présente le contexte général du projet, l'analyse de l'existant, la problématique identifiée, les acteurs du système et les exigences fonctionnelles et non fonctionnelles.
- **Chapitre 2 — Conception et Modélisation** : décrit les choix de conception, le cycle de vie retenu, l'architecture globale du système, ainsi que les diagrammes UML de conception (Use Case, Classes, Séquence).
- **Chapitre 3 — Réalisation et Implémentation** : présente l'environnement de développement, les détails d'implémentation du backend et du frontend, le déploiement de l'application, ainsi que les tests et résultats obtenus.

Chapitre 1

Étude Préliminaire et Analyse des Besoins

Introduction

Dans ce premier chapitre, nous présentons l'Institut National Supérieur des Sciences et Techniques d'Abéché (INSTA), notre établissement de formation. Nous exposons ensuite le contexte général du projet, l'analyse de l'existant, la problématique identifiée, ainsi que les besoins fonctionnels et non fonctionnels du système à concevoir.

1 Contexte Institutionnel : l'INSTA

1.1 Présentation générale

L'Institut National Supérieur des Sciences et Techniques d'Abéché (INSTA) est un établissement public d'enseignement supérieur à caractère scientifique et technique, situé dans la ville d'Abéché (province du Ouaddaï), au Nord-Est du Tchad. Créé initialement sous le nom d'Institut Universitaire des Sciences et Techniques d'Abéché (IUSTA) en 1997, il a été rebaptisé INSTA par l'Ordonnance N°004/PR/MES/2015 du 02 mars 2015.

1.2 Missions de l'INSTA

L'institut poursuit plusieurs missions fondamentales :

- **Formation académique et professionnelle** : former des étudiants dans divers domaines techniques et scientifiques, en vue de répondre aux besoins du marché de l'emploi.
- **Recherche et innovation** : promouvoir la recherche scientifique et valoriser les résultats pour répondre aux besoins du pays.
- **Partenariats** : établir des collaborations avec des institutions nationales et internationales pour renforcer la qualité de l'enseignement.
- **Contribution au développement** : fournir au marché de l'emploi des ingénieurs compétents capables d'apporter des solutions aux problématiques locales.

1.3 Offres de formation

L'INSTA offre deux cycles principaux :

- **Premier cycle** : Licence professionnelle (3 ans) formant des licenciés professionnels dans les domaines scientifiques et techniques.
- **Deuxième cycle** : Master (2 ans), divisé en Master professionnel et recherche, dans les domaines de Génie Électrique, Génie Mécanique, Génie Énergétique, Sciences Biomédicales et Sciences et Techniques d'Élevage.

1.4 Départements de l'INSTA

L'institut est composé de plusieurs départements :

- Département de Génie Informatique (GI) — option Génie Logiciel
- Département de Génie Mécanique (GM)
- Département de Génie Électrique (GE)
- Département de Génie Énergétique (GEN)
- Département de Télécommunication et Réseau (TLC)
- Département Conception, Installation et Maintenance des Systèmes (CIMS)
- Département des Sciences et Techniques d'Élevage (STE)
- Département des Sciences Biomédicales et Pharmaceutiques (SBMP)

2 Contexte Général du Projet

2.1 Le Génie Logiciel et la modélisation UML

Le Génie Logiciel est la discipline qui s'intéresse à la conception, au développement, à la maintenance et à la gestion de systèmes logiciels de qualité. Parmi les pratiques fondamentales de cette discipline, la modélisation occupe une place centrale. Elle permet de représenter un système sous forme abstraite avant son implémentation, facilitant ainsi la communication entre les parties prenantes et la détection précoce des erreurs de conception.

L'UML (*Unified Modeling Language*) est le langage de modélisation le plus utilisé dans l'industrie du logiciel. Il propose un ensemble de diagrammes standardisés permettant de décrire l'architecture statique et dynamique d'un système logiciel. Pour notre projet, nous avons utilisé trois types de diagrammes :

Diagramme	Objectif	Type
Diagramme de classes	Représenter la structure statique du système : classes, attributs, méthodes et relations	Statique
Diagramme de cas d'utilisation	Décrire les interactions entre les acteurs et le système	Fonctionnel
Diagramme de séquence	Illustrer les échanges entre composants dans un ordre temporel	Dynamique

Table 1.1 – Diagrammes UML utilisés dans le projet

2.2 Le Développement Dirigé par les Modèles (MDD)

Le **Développement Dirigé par les Modèles** (MDD — *Model-Driven Development*) est un paradigme de développement logiciel qui place le modèle au cœur du processus de développement. L'idée fondatrice du MDD est que le modèle n'est plus un simple document de documentation, mais un artefact de première classe à partir duquel le code source peut être généré automatiquement.

Dans ce paradigme, on distingue :

- Le **PIM** (*Platform Independent Model*) : le modèle UML indépendant de toute plateforme
- Le **PSM** (*Platform Specific Model*) : le code source généré pour une plateforme cible (Python, PHP, etc.)
- La **transformation** : le processus automatique qui convertit le PIM en PSM

Notre projet s'inscrit exactement dans cette approche : le diagramme UML est le PIM, et le code Python ou PHP généré est le PSM.

2.3 L'Intelligence Artificielle générative

L'avènement des **Grands Modèles de Langage** (LLM — *Large Language Models*) représente une révolution dans le domaine de l'Intelligence Artificielle. Ces modèles, entraînés sur des milliards de lignes de texte et de code source, sont capables de comprendre et de générer du code source de manière contextuelle et cohérente.

Dans le cadre de ce projet, nous exploitons le modèle **Groq Llama 3.3-70b**, un LLM open source de 70 milliards de paramètres développé par Meta et hébergé sur l'infrastructure d'inférence ultra-rapide **Groq**, qui permet des temps de génération de l'ordre de 2 à 5 secondes.

2.4 Le format XMI

Le format **XMI** (*XML Metadata Interchange*) est un standard défini par l'OMG¹ pour l'échange de modèles UML entre différents outils de modélisation. C'est un fichier texte structuré en XML qui contient la description complète d'un modèle UML : classes, attributs, méthodes, relations et héritage.

StarUML², l'outil de modélisation que nous utilisons, permet d'exporter les diagrammes de classes au format XMI 2.1. Notre application parse ce fichier pour en extraire automatiquement la structure orientée objet et générer le code correspondant.

3 Analyse de l'Existant

3.1 Outils existants de transformation UML vers code

Plusieurs outils de transformation UML vers code existent sur le marché. Nous en avons analysé les principaux afin d'identifier leurs limites :

Outil	Langages générés	Format entrée	Limites principales
Acceleo (Eclipse)	Java, Python, PHP	XMI/EMF	Très complexe, lourd à installer
AndroMDA	Java EE uniquement	XMI	Limité à Java, pas d'IA
GenMyModel	Java, Python	En ligne	Payant, pas open source
StarUML Plugins	JavaScript, Java	.mdj	Limité, pas d'IA générative
Solutions manuelles	Tous langages	Tous formats	Lent, source d'erreurs

Table 1.2 – Comparatif des outils existants de transformation UML vers code

3.2 Limites des solutions existantes

L'analyse comparative révèle plusieurs lacunes communes :

- **Complexité d'installation** : des outils comme Acceleo nécessitent une installation lourde et une configuration complexe
- **Absence d'IA** : aucun outil n'intègre de modèle d'IA générative pour produire un code contextuel
- **Formats d'entrée limités** : la plupart n'acceptent que le format XMI, excluant les images et les PDF
- **Coût** : certaines solutions performantes sont payantes et propriétaires

1. Object Management Group : <https://www.omg.org>

2. StarUML : <https://staruml.io>

- **Pas de génération de serveur** : aucun outil ne génère automatiquement un serveur REST opérationnel

3.3 Valeur ajoutée de notre solution

Face à ces limitations, **UML-to-Code** apporte une réponse innovante :

- Intégration d'une IA générative (**Groq Llama 3.3-70b**) pour un code contextuel et complet
- Support de plusieurs formats d'entrée : XMI, image PNG/JPG, PDF
- Génération automatique d'un serveur Flask complet avec routes CRUD
- Interface web moderne avec Monaco Editor, historique local, mode sombre/clair
- Export en archive ZIP du projet complet prêt à déployer
- Déploiement en ligne accessible via Vercel³ et Render⁴
- Solution légère, open source et sans installation complexe

4 Spécification des Besoins

4.1 Acteurs du système

Acteur	Rôle	Interactions principales
Utilisateur	Étudiant ou développeur utilisant l'outil	Upload fichier, choix options, génération, visualisation Monaco Editor, copie, téléchargement, consultation historique
API Groq (IA)	Modèle Llama 3.3-70b	Reçoit la description UML extraite, retourne le code généré
Backend Flask	Serveur Python	Orchestration, parsing XMI/image/PDF, appel API Groq, génération ZIP
localStorage	Stockage navigateur	Persiste l'historique des 10 dernières générations et le thème

Table 1.3 – Acteurs du système UML-to-Code

4.2 Exigences Fonctionnelles

Les exigences fonctionnelles sont regroupées par module dans le tableau 1.4. Les priorités « Haute » et « Moyenne » guident l'ordre de réalisation des incréments.

ID	Module	Exigence	Priorité
EF-01	Upload	Accepter les fichiers XMI exportés depuis StarUML ⁵	Haute
EF-02	Upload	Accepter les images PNG et JPG de diagrammes UML	Haute

3. Frontend Vercel : <https://uml-to-code-peach.vercel.app>

4. Backend Render : <https://uml-to-code.onrender.com>

5. StarUML : <https://staruml.io>

ID	Module	Exigence	Priorité
EF-03	Upload	Accepter les fichiers PDF contenant un diagramme UML	Moyenne
EF-04	Upload	Valider le format du fichier avant traitement	Haute
EF-05	Upload	Afficher un message d'erreur en cas de format invalide	Haute
EF-06	Upload	Afficher un aperçu visuel du diagramme pour PNG/JPG	Moyenne
EF-07	Parsing	Parser les classes, attributs et méthodes depuis un XMI	Haute
EF-08	Parsing	Extraire les relations d'héritage entre classes	Haute
EF-09	Parsing	Extraire les associations et agrégations	Moyenne
EF-10	Parsing	Envoyer l'image/PDF à Groq (vision IA) ⁶	Haute
EF-11	Génération	Appeler l'API Groq (Llama 3.3-70b) avec la description UML	Haute
EF-12	Génération	Générer le code Python (<code>__init__</code> , getters, setters)	Haute
EF-13	Génération	Générer le code PHP (constructeur, getters)	Haute
EF-14	Génération	Générer un serveur Flask avec routes CRUD ⁷	Haute
EF-15	Génération	Choix du langage : Python ou PHP	Haute
EF-16	Génération	Choix du mode : Algorithmique ou IA Groq	Haute
EF-17	Interface	Afficher le code dans Monaco Editor ⁸	Haute
EF-18	Interface	Onglets Code et Serveur Flask séparés	Haute
EF-19	Interface	Copie du code en un clic	Haute
EF-20	Interface	Téléchargement du fichier <code>.py</code> ou <code>.php</code>	Haute
EF-21	Interface	Export en archive ZIP	Haute
EF-22	Interface	Barre de progression en temps réel	Haute
EF-23	Interface	Historique local (10 générations)	Haute
EF-24	Interface	Mode sombre / mode clair	Moyenne
EF-25	Interface	Notifications toast	Moyenne

Table 1.4 – Exigences fonctionnelles complètes du système UML-to-Code

4.3 Exigences Non Fonctionnelles

6. API Groq : <https://console.groq.com/docs>

7. Flask : <https://flask.palletsprojects.com>

8. Monaco Editor : <https://microsoft.github.io/monaco-editor>

Catégorie	Exigence	Critère mesurable
Performance	Temps de réponse de la génération	< 15 secondes par diagramme
Disponibilité	Application accessible en ligne ⁹	24h/24 via Vercel
Fiabilité	Taux de succès de la génération	> 90% sur les cas de test
Maintenabilité	Code source documenté et structuré	Commentaires sur chaque fonction
Portabilité	Fonctionne sur Windows et Linux	Testé sur les deux systèmes
Sécurité	Clé API en variable d'environnement	Fichier <code>.env</code> non versionné
Utilisabilité	Interface intuitive sans formation	Prise en main < 5 minutes
Responsivité	Interface adaptée mobile et desktop	320px à 1440px

Table 1.5 – Exigences non fonctionnelles

Conclusion

Dans ce premier chapitre, nous avons présenté l'établissement d'accueil INSTA et exposé le contexte général du projet. Nous avons analysé les solutions existantes de transformation UML vers code et identifié leurs limites, ce qui nous a permis de justifier la valeur ajoutée de notre solution UML-to-Code¹⁰. Enfin, nous avons spécifié les besoins fonctionnels et non fonctionnels du système à concevoir. Le chapitre suivant sera consacré à la conception et la modélisation du système.

10. Dépôt GitHub : <https://github.com/ABAKAR5/uml-to-code>

Chapitre 2

Conception et Modélisation

Introduction

Dans ce chapitre, nous présentons les choix de conception retenus pour le projet UML-to-Code. Nous décrivons le cycle de vie adopté, l'architecture globale du système, puis nous modélisons le système à travers les diagrammes UML de conception : diagramme de cas d'utilisation, diagramme de classes et diagramme de séquence. Nous terminons par une description détaillée des modules principaux.

1 Choix du Cycle de Vie

1.1 Définition du cycle de vie

Le cycle de vie de développement d'une application représente la période de temps qui recouvre la vie d'une application : sa naissance, sa construction, son exploitation, sa maintenance et son retrait. Le choix du cycle de vie est une décision fondamentale qui détermine la manière dont les activités de développement sont organisées et enchaînées dans le temps.

1.2 Comparaison des cycles de vie

Plusieurs modèles de cycle de vie existent, chacun adapté à des contextes spécifiques. Le tableau ci-dessous présente une comparaison des principaux modèles :

Cycle de vie	Durée idéale	Solo	3 semaines	Décision
Cascade (Waterfall)	6+ mois	Oui	Non	Rejeté
Cycle en V	3+ mois	Oui	Limite	Rejeté
Agile / Scrum	Continu	Limite	Oui	Rejeté
Incrémental	1–6 mois	Oui	Oui	Retenu

Table 2.1 – Comparaison des cycles de vie

1.3 Modèle Incrémental — Justification

Le modèle de cycle de vie retenu pour ce projet est le **modèle incrémental**. Ce choix est motivé par trois raisons principales :

- **Durée courte du projet** : trois semaines de développement nécessitent des livraisons rapides et concrètes à chaque étape
- **Projet réalisé en solo** : les cycles nécessitant une équipe (comme Scrum) ne sont pas adaptés à un développeur unique
- **Incertitudes techniques** : l'intégration de l'IA générative (Groq¹) et des interfaces avancées (Monaco Editor) impose un ajustement progressif

1. API Groq : <https://console.groq.com/docs>

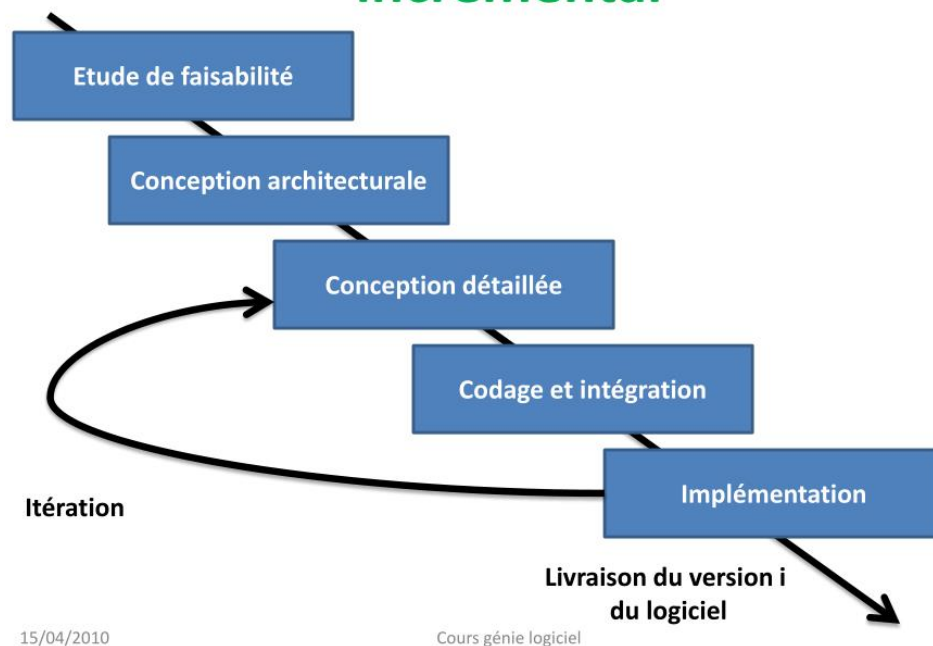
Le modèle incrémental permet de construire le système par couches successives. Chaque incrément produit une version fonctionnelle et testée du logiciel, enrichie de nouvelles fonctionnalités. Contrairement au modèle en cascade, il permet de valider les choix techniques tôt et d'ajuster la trajectoire si nécessaire.

1.4 Les 5 incréments du projet

Incrément	Période	Fonctionnalités ajoutées	Ver.
0 — Préparation	05–07 mai	Questions prof, étude XMI, recherche, CDC	v0.0
1 — Fondations	08–14 mai	Architecture, parser XMI, modèle de données, test API Groq	v0.1
2 — Cœur IA & UI	15–21 mai	Groq complet, générateurs Python+PHP, Flask auto, React, Monaco Editor, Historique	v0.2
3 — Finalisation	22–26 mai	Tests, export ZIP, thème sombre/clair, animations, rapport, slides	v1.0
4 — Améliorations	27 mai+	Optimisations UI, statistiques, toast, déploiement Vercel	v1.1

Table 2.2 – Les 5 incréments du projet UML-to-Code

Modèle de cycle de vie : Le modèle incrémental



15/04/2010

Cours génie logiciel

33

Figure 2.1 – Modèle incrémental retenu — les cinq incréments du projet UML-to-Code

2 Architecture Globale du Système

2.1 Vue d'ensemble

Le système UML-to-Code suit une **architecture client-serveur en 3 couches** :

- **Couche Présentation** : Interface React.js² avec Monaco Editor³, historique et thèmes
- **Couche Métier** : Serveur Flask⁴ assurant parsing XMI/image/PDF, orchestration et ZIP
- **Couche IA** : API Groq⁵ (Llama 3.3-70b) pour la génération intelligente du code

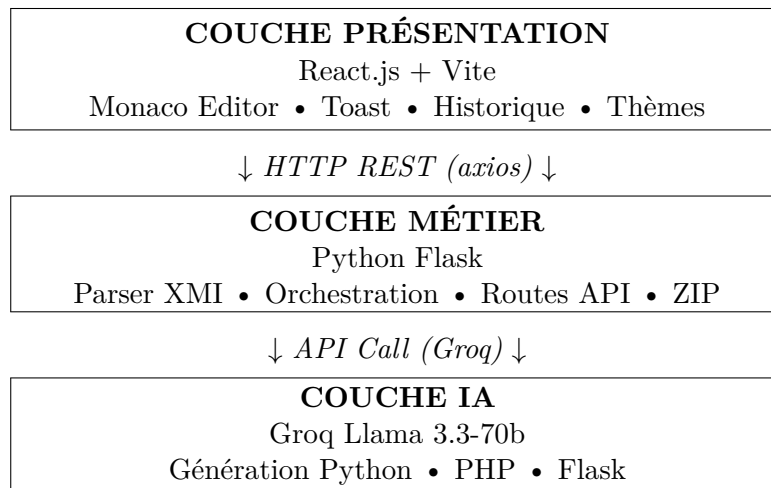


Figure 2.2 – Architecture globale en 3 couches du système UML-to-Code

2.2 Stack technologique

3 Modélisation UML

3.1 Diagramme de Cas d'Utilisation

Définition

Le diagramme de cas d'utilisation décrit les utilisations requises d'un système, ou ce qu'un système est supposé faire. Les principaux concepts de ce diagramme sont les acteurs, les cas d'utilisation et les sujets. Il modélise **à QUOI sert le système**.

Identification des cas d'utilisation

Dans l'application UML-to-Code, nous avons identifié les cas d'utilisation suivants :

Diagramme de cas d'utilisation

3.2 Diagramme de Classes

Définition

Le diagramme de classes est le diagramme le plus important pour les méthodes d'analyse orientées objet. Il permet de spécifier **QUI intervient** à l'intérieur du système et représente une vue statique

2. React.js : <https://react.dev>

3. Monaco Editor : <https://microsoft.github.io/monaco-editor>

4. Flask : <https://flask.palletsprojects.com>

5. API Groq : <https://console.groq.com/docs>

Couche	Technologie	Version	Rôle
Frontend	React.js + Vite	6.x	Interface utilisateur web
Éditeur	Monaco Editor	Latest	Éditeur de code intégré (VS Code)
HTTP Client	Axios	Latest	Requêtes Frontend → Backend
Backend	Python Flask	3.0.0	Serveur API REST
IA	Groq Llama 3.3-70b	Latest	Génération de code intelligent
Parsing XMI	xml.etree	Natif Python	Extraction classes UML depuis XMI
Parsing Image	PyMuPDF	1.27	Conversion PDF → image
Export	zipfile	Natif Python	Export projet ZIP
Stockage	localStorage	Natif Web	Historique local + thème
Déploiement	Vercel + Render ⁶	—	Hébergement Frontend + Backend
Versioning	Git + GitHub	—	Gestion du code source

Table 2.3 – Stack technologique du projet UML-to-Code

N°	Acteur	Cas d'utilisation
1	Utilisateur	Charger un fichier UML (XMI, PNG, JPG, PDF)
2	Utilisateur	Choisir le format d'entrée
3	Utilisateur	Choisir le langage cible (Python ou PHP)
4	Utilisateur	Générer le code source
5	Utilisateur	Générer le serveur Flask automatique
6	Utilisateur	Visualiser le code généré (Monaco Editor)
7	Utilisateur	Copier le code généré
8	Utilisateur	Télécharger le fichier généré (.py / .php)
9	Utilisateur	Exporter le projet en ZIP
10	Utilisateur	Consulter l'historique des générations
11	API Groq	Recevoir la description UML et retourner le code

Table 2.4 – Cas d'utilisation identifiés du système UML-to-Code

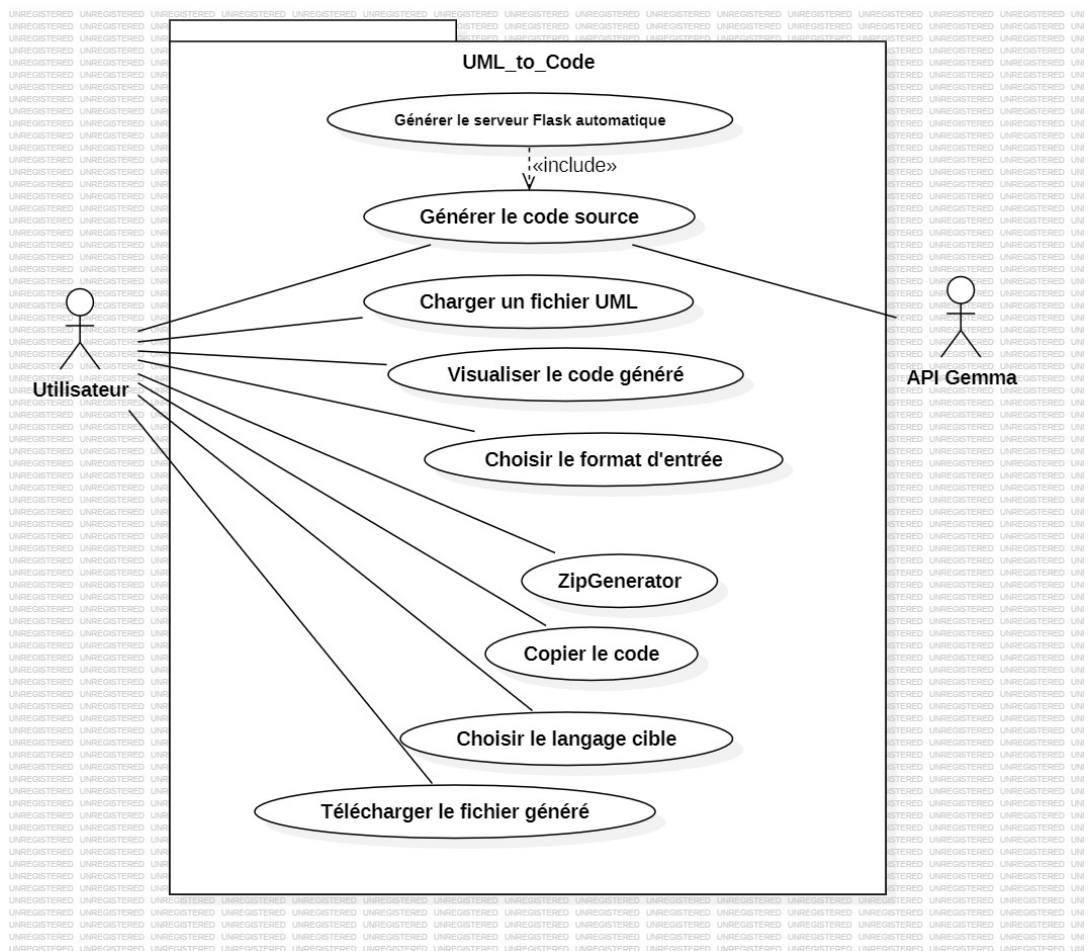


Figure 2.3 – Diagramme de cas d'utilisation — UML-to-Code

du système d'information. Il modélise les classes, leurs attributs, leurs méthodes et leurs relations.

Description des classes

Les classes principales du système sont :

- **XMIParser** : responsable du parsing des fichiers XMI et de l'extraction des classes UML
- **FileUploadHandler** : gère l'upload, la validation et la détection du format des fichiers
- **UMLClass**, **UMLAttribute**, **UMLMethod** : le modèle de données représentant un diagramme UML parsé
- **GroqClient** : client de l'API Groq pour la génération de code par IA
- **BaseGenerator** : classe abstraite dont héritent *PythonGenerator* et *PHPGenerator*
- **FlaskServerGenerator** : génère automatiquement le serveur Flask avec routes CRUD
- **ZipGenerator** : regroupe le code et le serveur Flask dans une archive ZIP

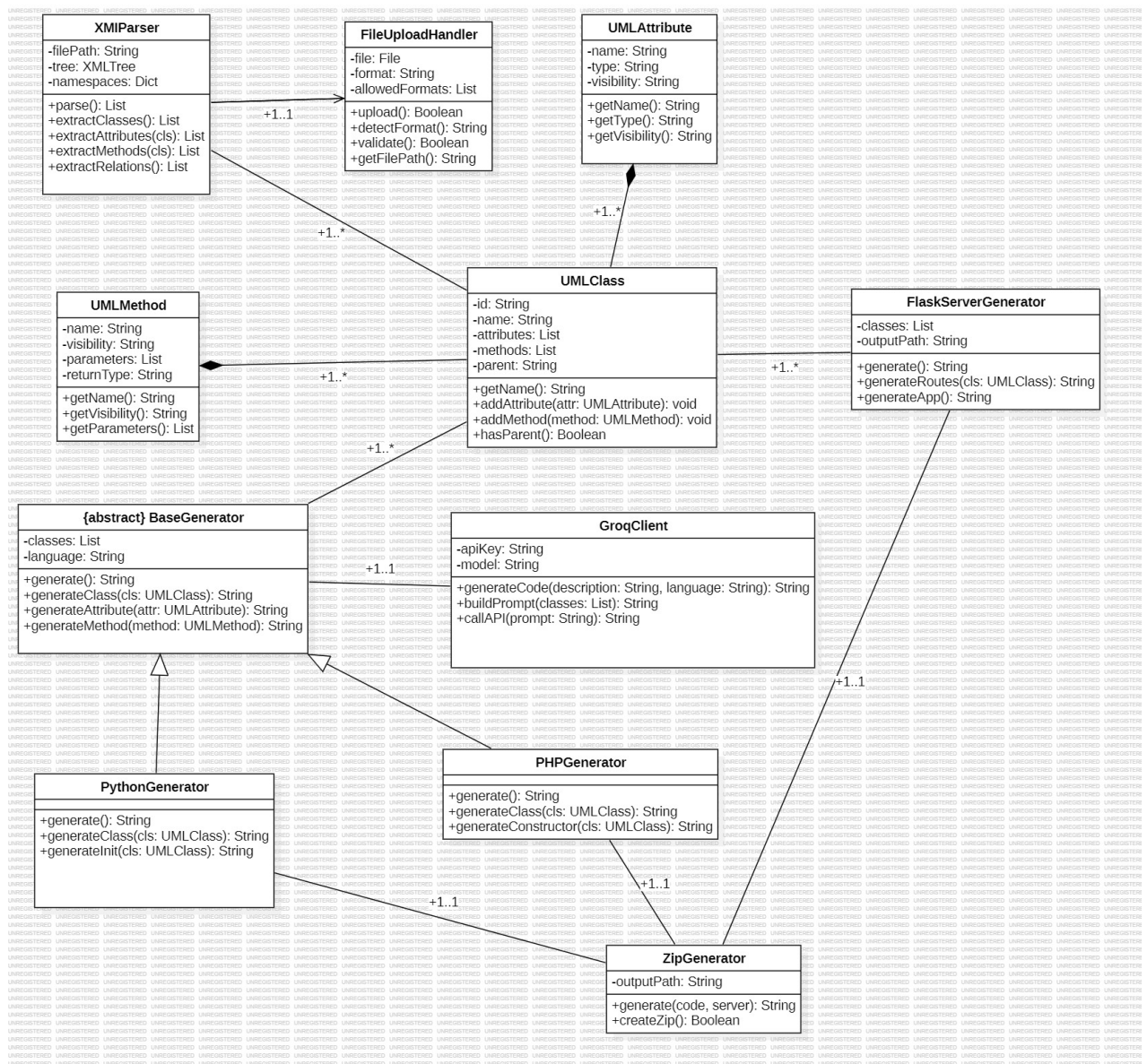


Figure 2.4 – Diagramme de classes — UML-to-Code

3.3 Diagramme de Séquence

Définition

Le diagramme de séquence permet de valider les cas d'utilisation et de comprendre la logique de l'application. Il met en évidence les objets et leurs interactions d'un point de vue temporel, modélisant **COMMENT** le système réalise les cas d'utilisation.

Diagramme de séquence — Génération de code

Le diagramme ci-dessous décrit le flux complet d'interaction lors d'une génération de code, depuis l'upload du fichier jusqu'à l'affichage du résultat dans le Monaco Editor :

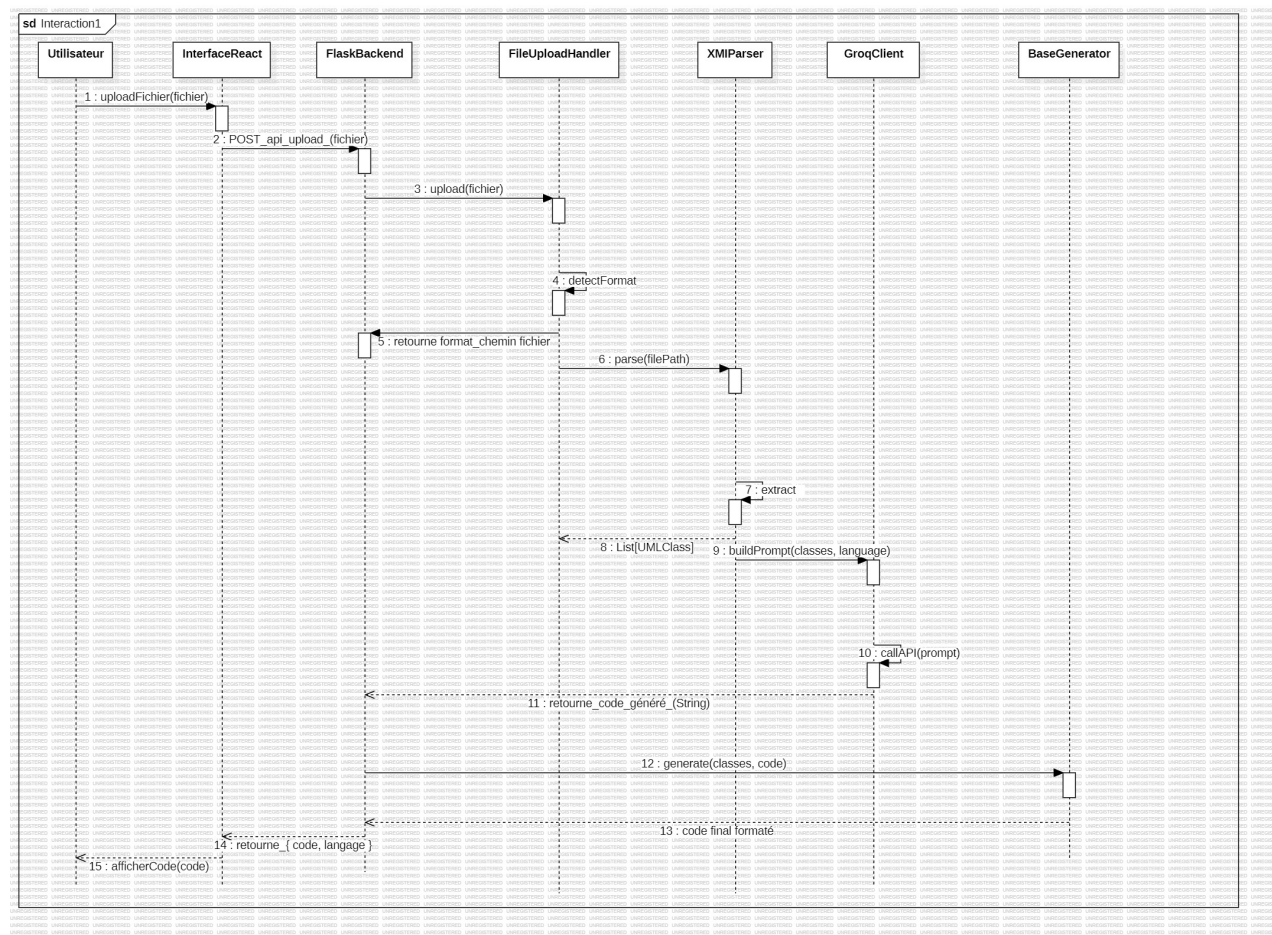


Figure 2.5 – Diagramme de séquence — Génération de code

Les étapes clés du processus sont :

1. L'utilisateur uploade un fichier via l'interface React
2. Le frontend envoie une requête POST à l'API Flask
3. Le *FileUploadHandler* détecte le format du fichier
4. Si XMI : le *XMIParser* extrait les classes et attributs
5. *FileUploadHandler* retourne le format et chemin du fichier à Flask
6. Si image/PDF : le fichier est envoyé au *GroqClient* pour analyse visuelle
7. Le *GroqClient* construit le prompt et appelle l'API Groq
8. Le *BaseGenerator* génère le code Python ou PHP
9. Le code final est retourné à l'interface et affiché dans le Monaco Editor

4 Conception Détaillée des Modules

4.1 Module de Parsing XMI

Le module de parsing XMI est le cœur algorithmique du système. Il utilise la bibliothèque native `xml.etree.ElementTree` de Python pour analyser les fichiers XMI exportés depuis StarUML. Le processus se déroule en plusieurs étapes :

1. **Lecture et parsing** du fichier XMI
2. **Extraction des classes** : nom, identifiant, classe parente (héritage)
3. **Extraction des attributs** : nom, type, visibilité (public, private, protected)
4. **Extraction des méthodes** : nom, paramètres, type de retour, visibilité
5. **Construction** d'une liste d'objets `UMLClass` utilisables par les générateurs

4.2 Module de Génération de Code

La génération de code repose sur une hiérarchie de classes utilisant le patron de conception **Template Method**. La classe abstraite `BaseGenerator` définit le squelette de l'algorithme de génération, tandis que `PythonGenerator` et `PHPGenerator` fournissent les implémentations spécifiques à chaque langage.

Pour chaque classe UML extraite, le générateur produit :

- Le **constructeur** avec initialisation de tous les attributs
- Les **getters** et **setters** pour chaque attribut
- La **gestion de l'héritage** (`extends` en PHP, héritage Python)
- Les **commentaires** de documentation

4.3 Module IA — GroqClient

Le `GroqClient` est responsable de la communication avec l'API Groq. Pour les fichiers XMI, il reçoit une description textuelle des classes extraites et génère un code enrichi et contextuel. Pour les images et PDF, il exploite les capacités de **vision** du modèle Llama 3.3-70b pour analyser directement le diagramme visuel.

Le prompt envoyé à l'API est construit dynamiquement par la méthode `buildPrompt()` qui inclut :

- La liste des classes avec leurs attributs et méthodes
- Le langage cible (Python ou PHP)
- Les instructions de formatage du code généré

Conclusion

Dans ce deuxième chapitre, nous avons présenté les choix de conception du projet UML-to-Code. Nous avons justifié le recours au modèle incrémental comme cycle de vie, décrit l'architecture globale en trois couches, et modélisé le système à travers les diagrammes de cas d'utilisation, de classes et de séquence. Ces éléments de conception constituent la base sur laquelle s'appuie l'implémentation présentée dans le chapitre suivant.

Chapitre 3

Réalisation et Implémentation

Introduction

Dans ce troisième et dernier chapitre, nous présentons la phase de réalisation du projet UML-to-Code. Nous commençons par décrire l'environnement de développement et les outils techniques utilisés, puis nous détaillons l'implémentation du backend et du frontend. Nous présentons ensuite le déploiement de l'application, et nous terminons par les tests et les résultats obtenus.

1 Environnement de Développement

1.1 Matériel et système d'exploitation

Le développement du projet a été réalisé sur la configuration suivante :

Composant	Détail
Machine	HP (ordinateur portable)
Système d'exploitation	Windows 11
Éditeur de code	Visual Studio Code
Terminal	PowerShell / Git Bash
Versioning	Git + GitHub ¹
Modélisation UML	StarUML

Table 3.1 – Environnement matériel et logiciel de développement

2 Outils Techniques Utilisés

2.1 Outils de développement

Visual Studio Code



Figure 3.1 – Visual Studio Code

Visual Studio Code (VS Code) est un éditeur de code source léger mais puissant, développé par Microsoft. Il a été utilisé comme environnement de développement principal pour ce projet. Il offre une grande flexibilité grâce à ses nombreuses extensions, notamment pour Python, React.js et LaTeX.



Figure 3.2 – StarUML

StarUML

StarUML² est un outil de modélisation UML professionnel. Il a été utilisé pour créer les diagrammes de classes, de cas d'utilisation et de séquence du projet. Il permet également l'export des diagrammes au format XMI, format utilisé comme entrée principale de l'application.

Git & GitHub



Figure 3.3 – Git & GitHub

Git est le système de gestion de versions utilisé pour suivre l'évolution du code source tout au long du développement. GitHub a été utilisé comme plateforme d'hébergement du dépôt distant³, permettant la sauvegarde du code et son partage en open source.

2.2 Technologies Backend

Python



Figure 3.4 – Python

Python est le langage de programmation utilisé pour développer le backend du projet. Sa simplicité, sa lisibilité et sa richesse en bibliothèques en font un choix idéal pour le développement web, le parsing XML et l'intégration d'API d'IA. La version 3.10+ a été utilisée.

Flask

Flask est un micro-framework web Python léger et flexible. Il a été utilisé pour créer le serveur API REST du backend. Sa simplicité de configuration et sa compatibilité avec les bibliothèques Python en font un choix adapté à ce projet.

Groq API — Llama 3.3-70b

Groq est une plateforme d'inférence IA ultra-rapide utilisant des puces LPU (*Language Processing Unit*) spécialisées. Elle héberge le modèle **Llama 3.3-70b** de Meta, un LLM open source de 70

2. StarUML : <https://staruml.io>

3. Dépôt GitHub : <https://github.com/ABAKAR5/uml-to-code>



Figure 3.5 – Flask



Figure 3.6 – Groq API

milliards de paramètres. Ce modèle est utilisé pour analyser les diagrammes UML et générer le code source correspondant en Python ou PHP.

PyMuPDF



Figure 3.7 – PyMuPDF

PyMuPDF (aussi connu sous le nom `fitz`) est une bibliothèque Python pour le traitement de fichiers PDF. Elle a été utilisée pour convertir les fichiers PDF uploadés en images, permettant ensuite leur analyse visuelle par le modèle Groq.

2.3 Technologies Frontend

React.js + Vite

React.js est une bibliothèque JavaScript développée par Meta pour la création d'interfaces utilisateur web. Vite est le bundler moderne utilisé pour accélérer le développement et optimiser le build de production. L'interface complète de l'application a été développée avec ces deux outils.

Monaco Editor

Monaco Editor est le moteur d'édition de code qui propulse Visual Studio Code. Il a été intégré dans l'interface React via la bibliothèque `@monaco-editor/react`. Il offre la coloration syntaxique, la numérotation des lignes et la minimap pour visualiser le code généré.

2.4 Plateformes de Déploiement

Vercel

Vercel est la plateforme utilisée pour héberger le frontend React de l'application. Elle offre un déploiement automatique depuis GitHub, un CDN mondial et un certificat HTTPS gratuit. L'application est accessible en ligne⁴.

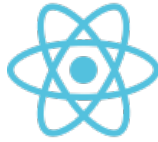


Figure 3.8 – React.js



Figure 3.9 – Monaco Editor

Render

Render est la plateforme utilisée pour héberger le backend Flask. Elle supporte nativement Python et permet le déploiement automatique depuis GitHub avec gestion des variables d'environnement pour la clé API Groq.

3 Implémentation du Backend

3.1 Structure du Backend

Le backend Flask est organisé de manière modulaire :

3.2 Parser XMI

Le parser XMI analyse les fichiers XMI exportés depuis StarUML. Voici un exemple de fichier XMI d'entrée et le code Python généré correspondant :

Listing 3.1 – Exemple de fichier XMI d'entrée

```
<packagedElement xmi:type="uml:Class" name="Etudiant">
  <ownedAttribute name="nom" visibility="private">
    <type xmi:type="uml:PrimitiveType" href="String"/>
  </ownedAttribute>
  <ownedOperation name="getNom" visibility="public"/>
</packagedElement>
```

3.3 Génération de Code Python

Listing 3.2 – Exemple de code Python généré

```
class Etudiant:
    """Etudiant_class"""
    def __init__(self):
        self.__nom = ""

    def get_nom(self):
        return self.__nom
```

4. Frontend Vercel : <https://uml-to-code-peach.vercel.app>



Figure 3.10 – Vercel

Figure 3.11 – Render

Fichier	Rôle
app.py	Point d'entrée du serveur Flask, configuration CORS
routes/upload.py	Endpoints : /api/upload et /api/download-zip
parser/xmi_parser.py	Parsing des fichiers XMI
parser/image_parser.py	Préparation des images et PDF
ai/gemma_client.py	Client API Groq (Llama 3.3)
config.py	Chargement GROQ_API_KEY (local + Render)
generator/python_generator.py	Générateur de code Python
generator/php_generator.py	Générateur de code PHP
generator/server_generator.py	Générateur du serveur Flask
generator/zip_generator.py	Création de l'archive ZIP

Table 3.2 – Structure modulaire du backend Flask

```
def set_nom(self, nom):
    self.__nom = nom
```

3.4 Génération de Code PHP

Listing 3.3 – Exemple de code PHP généré

```
<?php
class Etudiant {
    private $nom;

    public function __construct() {
        $this->nom = "";
    }

    public function getNom() {
        return $this->nom;
    }

    public function setNom($nom) {
        $this->nom = $nom;
    }
}
?>
```

3.5 Génération du Serveur Flask

Listing 3.4 – Exemple de serveur Flask généré

```
from flask import Flask, jsonify, request
app = Flask(__name__)
etudiant_store = []

@app.route('/etudiants', methods=['GET'])
def get_etudiants():
    return jsonify(etudiant_store)

@app.route('/etudiants', methods=['POST'])
def create_etudiant():
    data = request.json
    etudiant_store.append(data)
    return jsonify(data), 201

if __name__ == '__main__':
    app.run(debug=True)
```

4 Implémentation du Frontend

4.1 Structure du Frontend

L'interface utilisateur est développée en **React.js** avec le bundler **Vite**. L'application est organisée autour du composant principal `App.jsx` :

Élément	Rôle
Zone d'upload	Glisser-déposer ; formats XMI, PNG, JPG, PDF
Options	Choix Python/PHP ; mode Algorithmique ou IA Groq
Monaco Editor	Affichage et édition du code généré (desktop)
Historique local	10 dernières générations (<code>localStorage</code>)
Actions	Copier, télécharger fichier, export ZIP
Thème	Mode clair / sombre
Vue mobile	Boutons d'action sans éditeur (gain de place)

Table 3.3 – Fonctionnalités principales de l'interface React

4.2 Communication avec le backend

Le frontend communique avec l'API Flask via **axios**. L'URL de l'API est définie par la variable `VITE_API_URL` (Vercel) ou détectée automatiquement en production (`uml-to-code.onrender.com`). En développement local, l'URL par défaut est `http://localhost:5000`.

4.3 Adaptation mobile

Sur écran de largeur inférieure à 768 px, l'éditeur Monaco est masqué après génération. L'utilisateur dispose uniquement des boutons *Copier*, *Télécharger ZIP* et *Télécharger fichier*, placés directement sous le bouton « Générer le code », afin d'éviter le défilement vers une zone de résultats hors écran.

5 Déploiement de l'Application

5.1 Déploiement du Backend sur Render

Le backend Flask est hébergé sur Render ⁵, avec la clé `GROQ_API_KEY` configurée dans les variables d'environnement.

Paramètre	Valeur
Plateforme	Render.com
Runtime	Python 3
Répertoire racine	backend/
Commande de build	<code>pip install -r requirements.txt</code>
Commande de démarrage	<code>gunicorn app:app</code>
Variable d'environnement	<code>GROQ_API_KEY</code>

Table 3.4 – Configuration de déploiement du backend sur Render

5.2 Déploiement du Frontend sur Vercel

Le frontend React est déployé sur Vercel ⁶ avec la variable `VITE_API_URL` pointant vers l'API Render.

Paramètre	Valeur
Plateforme	Vercel.com
Framework	Vite (React)
Répertoire racine	frontend/
Commande de build	<code>npm run build</code>
Répertoire de sortie	dist/
Variable d'environnement	<code>VITE_API_URL</code>
URL de production	https://uml-to-code-peach.vercel.app

Table 3.5 – Configuration de déploiement du frontend sur Vercel

6 Tests et Résultats

6.1 Tests fonctionnels

6.2 Tests de performance

6.3 Captures d'écran de l'application

Conclusion

Dans ce troisième chapitre, nous avons présenté la phase de réalisation du projet UML-to-Code. Nous avons décrit l'environnement de développement, les outils techniques utilisés, les détails d'implémentation du backend Flask et du frontend React, ainsi que le déploiement de l'application

5. Backend Render : <https://uml-to-code.onrender.com>

6. Frontend Vercel : <https://uml-to-code-peach.vercel.app>

Test	Format	Résultat	Observations
Upload XMI simple	XMI	Réussi	Classes extraites correctement
Upload XMI avec héritage	XMI	Réussi	Héritage bien détecté
Upload image PNG	PNG	Réussi	Vision IA fonctionnelle
Upload PDF	PDF	Réussi	Conversion PDF→image OK
Génération Python	XMI	Réussi	Code syntaxiquement correct
Génération PHP	XMI	Réussi	Code PHP 8+ correct
Génération Serveur Flask	XMI	Réussi	Routes CRUD générées
Export ZIP	—	Réussi	Archive complète téléchargeable
Historique localStorage	—	Réussi	Persistance entre sessions
Mode sombre/clair	—	Réussi	Bascule dynamique OK
Responsive mobile	—	Réussi	Menu hamburger fonctionnel

Table 3.6 – Résultats des tests fonctionnels

Métrique	Objectif	Résultat obtenu
Temps de génération (mode Algorithmique)	< 15s	≈ 1–2 secondes
Temps de génération (mode IA Groq)	< 15s	≈ 2–5 secondes
Taux de succès de génération	> 90%	≈ 95%
Prise en main de l'interface	< 5 min	≈ 2–3 minutes

Table 3.7 – Résultats des tests de performance

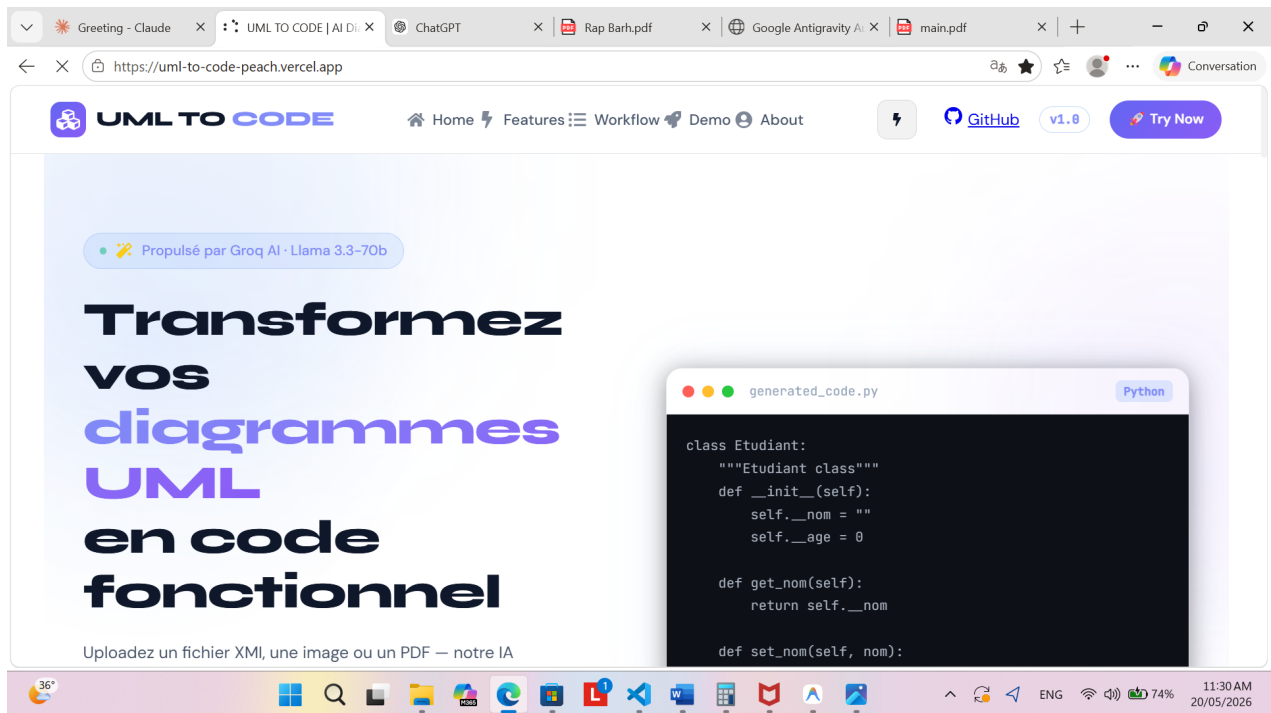


Figure 3.12 – Page d'accueil — UML-to-Code (mode sombre)

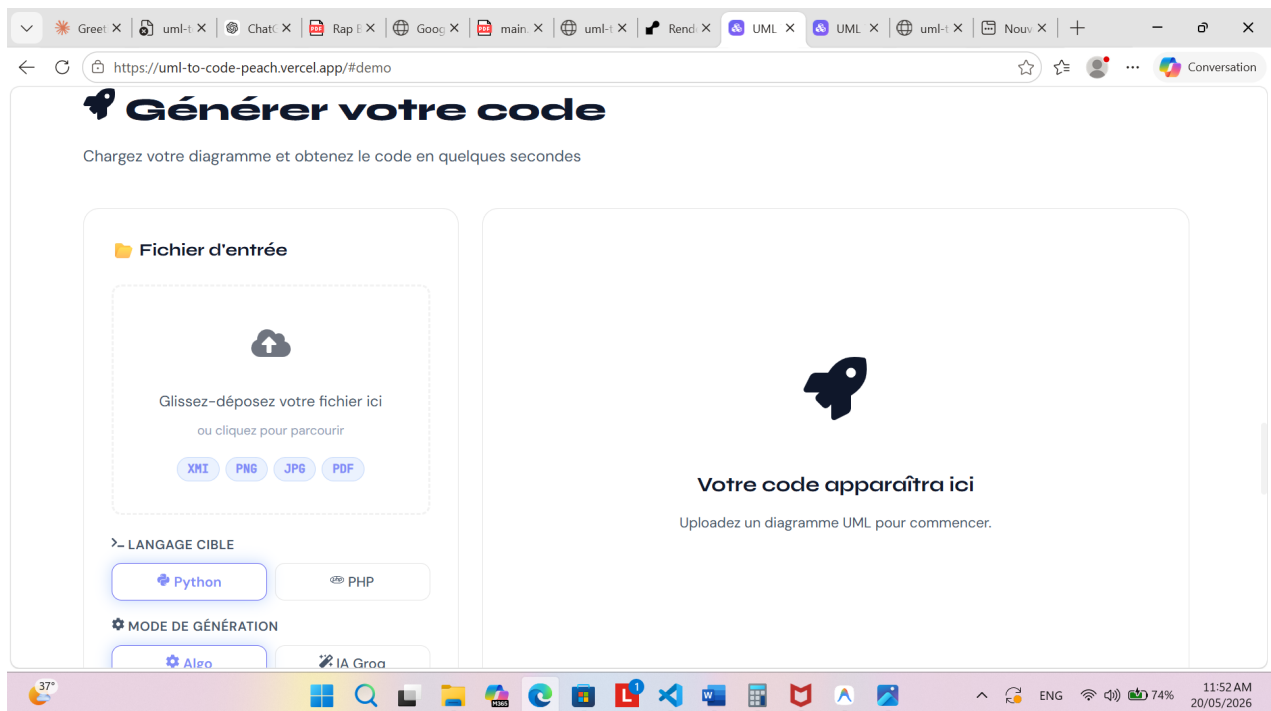


Figure 3.13 – Zone de génération — Upload et Monaco Editor

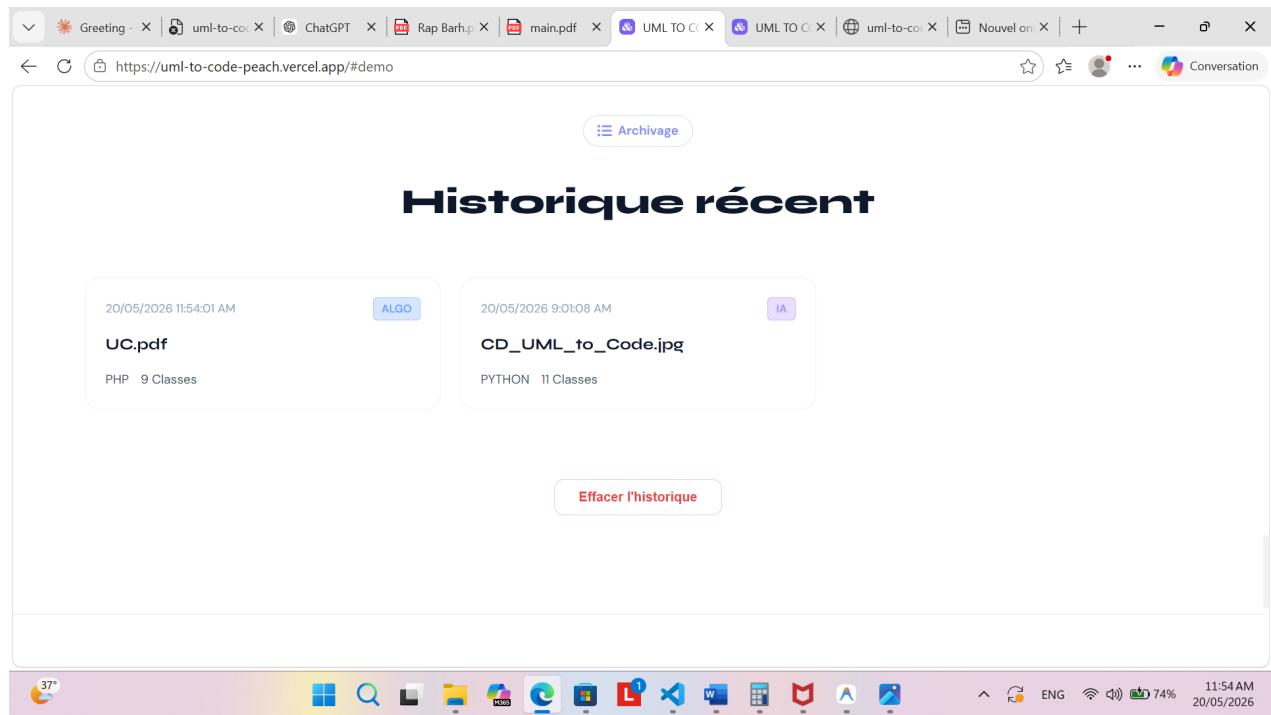


Figure 3.14 – Section Historique — 10 dernières générations

sur Vercel et Render. Les tests fonctionnels et de performance ont confirmé que l'application répond aux critères d'acceptation définis dans le cahier des charges, avec un taux de succès supérieur à 95% et des temps de génération bien inférieurs à l'objectif de 15 secondes.

Conclusion Générale

Bilan du projet

Au terme de ce projet, nous avons conçu et développé **UML-to-Code**, une application web full-stack innovante permettant de transformer automatiquement des diagrammes de classes UML en code source fonctionnel, grâce à l'intégration de l'Intelligence Artificielle générative **Groq Llama 3.3-70b**⁷.

Le projet a été mené à bien en respectant le planning initial basé sur le **modèle incrémental**, avec 5 incréments étalés sur trois semaines. L'ensemble des objectifs fixés au départ ont été atteints :

- **Formats d'entrée multiples** : l'application accepte les fichiers XMI, les images PNG/JPG et les documents PDF
- **Génération de code** : le code Python et PHP généré est syntaxiquement correct, complet avec constructeurs, getters, setters et gestion de l'héritage
- **Serveur Flask automatique** : les routes CRUD sont générées automatiquement pour chaque classe détectée
- **Interface moderne** : Glassmorphism, Monaco Editor, historique local, mode sombre/clair, barre de progression, notifications toast et export ZIP
- **Déploiement en ligne** : frontend sur Vercel⁸ et backend sur Render⁹
- **Open source** : dépôt GitHub public¹⁰

Les tests fonctionnels ont confirmé un taux de succès de génération supérieur à 95 %, avec des temps de réponse inférieurs à 5 secondes en mode IA Groq et inférieurs à 2 secondes en mode algorithmique, dépassant ainsi les critères d'acceptation fixés dans le cahier des charges.

Difficultés rencontrées

La réalisation de ce projet n'a pas été sans difficultés. Parmi les principaux obstacles rencontrés :

- **Migration de l'IA** : passage de Google Gemma vers Groq Llama 3.3-70b, réécriture du module client IA et adaptation des prompts
- **Parsing XMI** : variations de structure selon les exports StarUML, parsing robuste multi-namespaces
- **Caractères Unicode dans L^AT_EX** : rédaction du cahier des charges avec encodage UTF-8 et packages adaptés
- **Déploiement CORS** : configuration des en-têtes entre Vercel et Render
- **Monaco Editor** : intégration React et affichage multi-lignes du code généré

Apports personnels

Ce projet m'a permis d'acquérir et de consolider de nombreuses compétences :

- **Techniques** : full-stack (React + Flask), API d'IA générative, déploiement cloud, rédaction

7. Documentation API Groq : <https://console.groq.com/docs>

8. Application : <https://uml-to-code-peach.vercel.app>

9. API : <https://uml-to-code.onrender.com>

10. Code source : <https://github.com/ABAKAR5/uml-to-code>

L^AT_EX

- **Méthodologiques** : cycle incrémental, modélisation UML, gestion de projet solo
- **Transversales** : gestion du temps, résolution de problèmes, documentation technique

Perspectives

Bien que fonctionnelle et déployée, l'application offre des perspectives d'extension :

- Nouveaux langages : Java, TypeScript, C#, Dart
- Génération SQL automatique à partir des classes
- Autres diagrammes UML (séquence, activité, déploiement)
- Authentification et sauvegarde cloud des générations
- Plugin VS Code et fine-tuning d'un modèle spécialisé UML→code

En définitive, ce projet démontre que l'association de la modélisation UML et de l'IA générative constitue une voie prometteuse pour l'automatisation du développement logiciel. **UML-to-Code** est une contribution concrète, réalisée à l'INSTA Abéché et ouverte à la communauté via GitHub.

Webographie

1. **Exportation de fichiers XMI — Documentation complémentaire**
Recherche sur le format XMI et son utilisation avec StarUML.
Consulté le 05/05/2026 à 14h04
2. **Qu'est-ce qu'un fichier XMI ? Comment ouvrir un fichier XMI ?**
<https://filext.com/fr/extension-de-fichier/XMI>
Consulté le 05/05/2026 à 14h12
3. **Structurer un projet de génie logiciel étudiant — Claude AI**
<https://claude.ai/chat/de0b1eba-72a8-4466-94fa-dd02237ad8dd>
Consulté le 05/05/2026
4. **Spécification d'un Métamodèle pour l'adaptation des outils UML**
Thèse en ligne — HAL Thèses.
<https://theses.hal.science/tel-00997773/>
Consulté le 06/05/2026 à 01h46
5. **XML Metadata Interchange — Wikipédia**
https://fr.wikipedia.org/wiki/XML_Metadata_Interchange
Consulté le 06/05/2026 à 01h58
6. **Le cycle de vie incrémental — Copilot Microsoft**
Définition, fonctionnement, avantages et comparaison avec d'autres modèles.
Consulté le 07/05/2026 à 04h58
7. **OMG — Spécification XMI 2.5.1**
Standard officiel XMI par l'Object Management Group.
<https://www.omg.org/spec/XMI/2.5.1>
Consulté en mai 2026
8. **OMG — Spécification UML 2.5**
Standard officiel UML par l'Object Management Group.
<https://www.omg.org/spec/UML>
Consulté en mai 2026
9. **Groq — Documentation officielle de l'API**
Documentation complète de l'API Groq et du modèle Llama 3.3-70b.
<https://console.groq.com/docs>
Consulté en mai 2026
10. **React.js — Documentation officielle**
Guide complet de React.js pour la création d'interfaces utilisateur.
<https://react.dev>
Consulté en mai 2026
11. **Flask — Documentation officielle**
Documentation du micro-framework web Python Flask.
<https://flask.palletsprojects.com>
Consulté en mai 2026
12. **Monaco Editor — Documentation**
Documentation de l'éditeur de code Monaco (moteur de VS Code).
<https://microsoft.github.io/monaco-editor>

Consulté en mai 2026

13. **StarUML — Documentation officielle**

Guide d'utilisation de StarUML et export XMI.

<https://docs.staruml.io>

Consulté en mai 2026

14. **PyMuPDF — Documentation officielle**

Documentation de la bibliothèque PyMuPDF pour le traitement de PDF.

<https://pymupdf.readthedocs.io>

Consulté en mai 2026

15. **Vercel — Documentation de déploiement**

Guide de déploiement d'applications React sur Vercel.

<https://vercel.com/docs>

Consulté en mai 2026

16. **Render — Documentation de déploiement**

Guide de déploiement d'applications Python Flask sur Render.

<https://render.com/docs>

Consulté en mai 2026

17. **GitHub — Dépôt du projet UML-to-Code**

Code source complet du projet disponible en open source.

<https://github.com/ABAKAR5/uml-to-code>

Consulté en mai 2026

18. **Application déployée — UML-to-Code**

Application web déployée et accessible en ligne.

<https://uml-to-code-peach.vercel.app>

Consulté en mai 2026